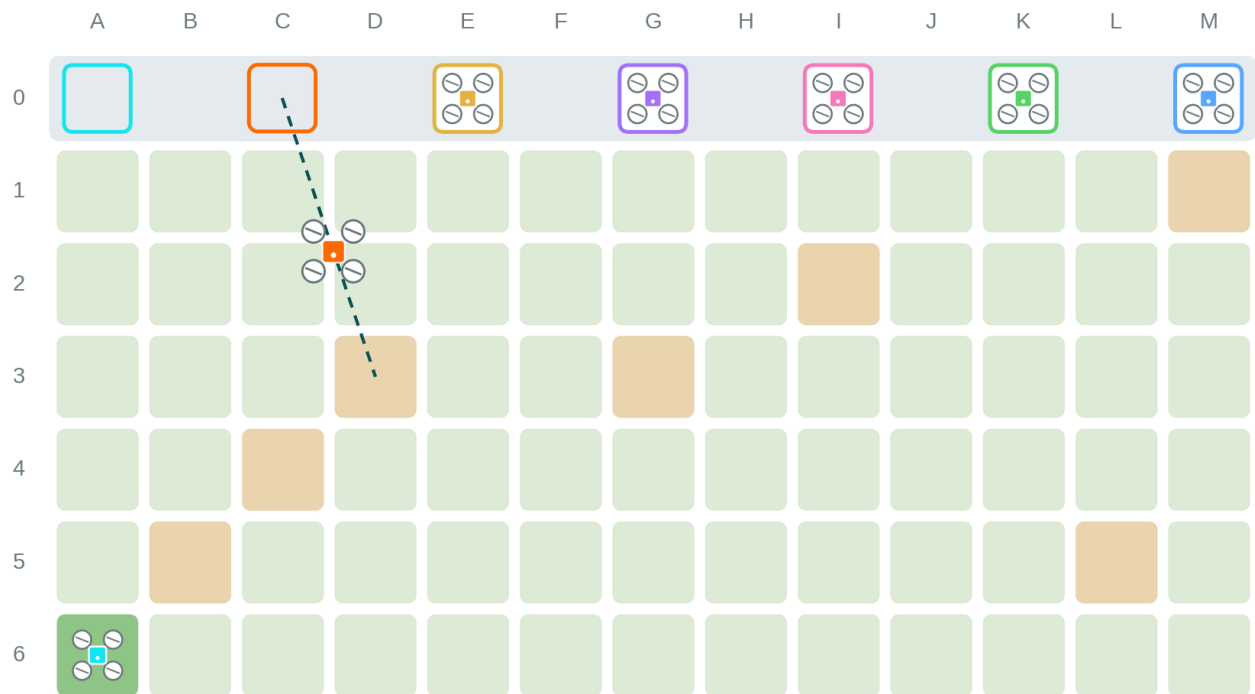


PRACTICAL MOCK EXAM

Farm Drones

Information Technology Paper I (Java)
Object-oriented programming



LANGUAGE
Java

AUTHOR
Kheagen Haskins

PASS CONDITION
Water every field on the farm

Instructions

1. This paper has five questions. Answer all of them, in order. Each question builds on the ones before it.
2. Answer the questions using object-oriented programming principles. Your program must make sensible use of methods and parameters.
3. Write your code so that it works with any data, not only the sample data in the files or in this paper. Look at the data files carefully before you start.
4. Develop any searches from first principles. Do not use the built-in features of Java to do them for you.
5. Refer to files by their names only, such as `drones.txt`. Do not use full path names.
6. If a section of your code will not work, comment it out so the rest of your program still runs, and add a comment explaining what went wrong.
7. The `exam.internal` package is provided for you. Do not change any file in it.

There are no marks. You pass when your drones water every field on the farm, and the program tells you when you have.

Files provided

File	Purpose
<code>drones.txt</code>	One line per drone in the fleet
<code>jobs.txt</code>	One line per field that needs water
<code>exam.internal</code>	The provided package: the <code>Drone</code> class you extend, the <code>Manager</code> class you extend, and the display that draws your drones

Scenario

A farm is laid out as a grid of fields. Columns are labelled with letters from `A`, and rows are numbered from `1`. A location on the farm is written as a column letter followed by a row number, such as `F12`.

Above row `1` is row `0`, where the farm's drones are kept in their docking bays. A drone parked in the bay above column `C` is at location `C0`.

Every drone carries a tank of water. When a field needs watering, the farm sends a drone from wherever it is to that field, and the drone releases the water there. It then stays at that field until it is sent somewhere else. Flying uses battery, and an older drone, or one carrying more water, uses more battery for the same distance. A drone that runs out of battery on the way falls out of the sky.

Your program will read the fleet of drones and the list of fields that need water, then decide which drone waters which field. The display then plays your decisions back as an animation, so you can watch every flight and see exactly where anything went wrong.

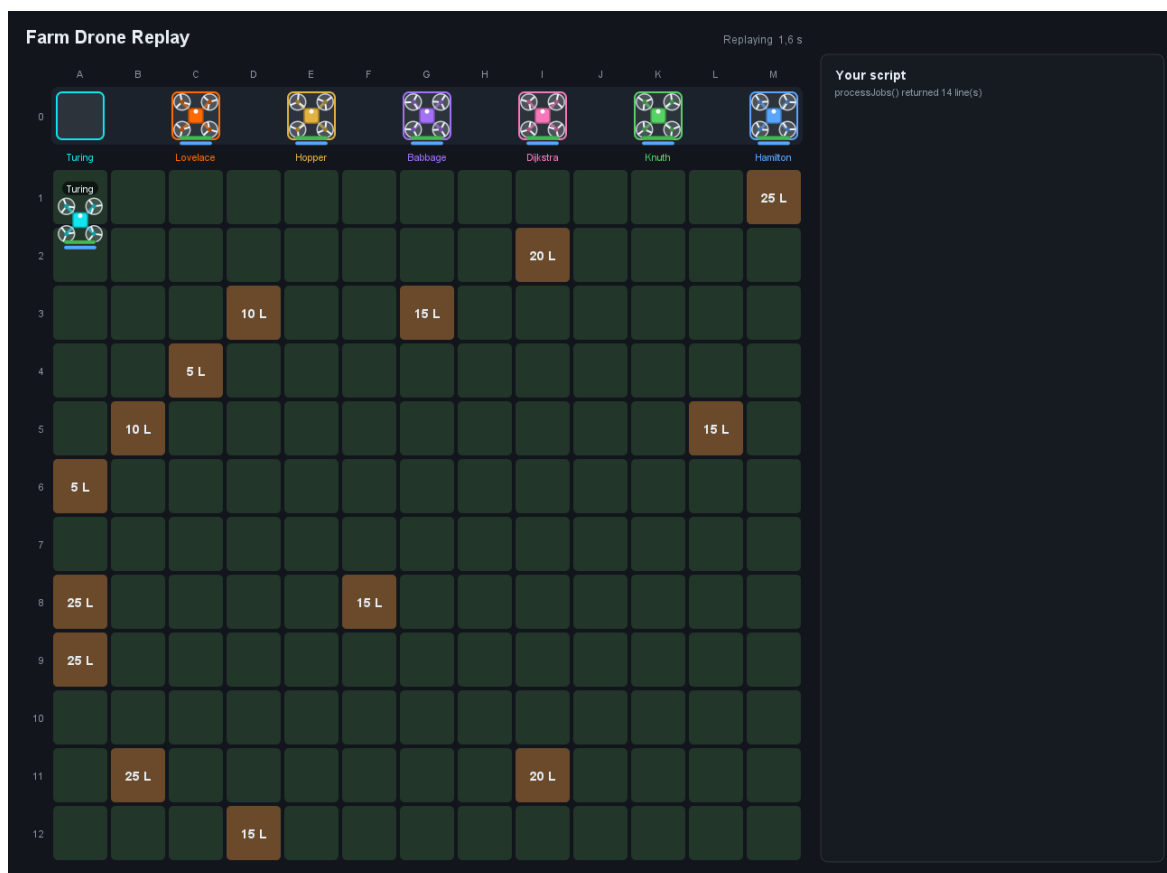


Figure 1: The farm display during a replay. The docking bays are in row 0, and each brown field shows the litres of water it needs.

The provided Drone class

`Drone` is an abstract class in the `internal` package. Every method in it throws an `UnsupportedOperationException` until your subclass overrides it, so your class compiles as soon as you write `extends Drone`, and you can build it one method at a time.

Method	Returns
<code>getName()</code>	<code>String</code> : the drone's name
<code>getPayload()</code>	<code>double</code> : the litres of water on board
<code>getBatteryLife()</code>	<code>double</code> : the battery percentage, from 100 down
<code>getPurchaseDate()</code>	<code>LocalDate</code> : the date the drone was bought
<code>getColourHex()</code>	<code>String</code> : the drone's colour as a hex string, such as <code>#12E5F0</code>
<code>getLocation()</code>	<code>String</code> : the drone's current location, such as <code>C0</code>

Method	Returns
<code>calculateBatteryDrain(String location)</code>	double: the battery percentage a flight to <code>location</code> would use
<code>deliverPayload(String location, double payload)</code>	String: flies the drone to <code>location</code> , releases the water, and describes the delivery

One litre of water weighs one kilogram, so the payload is both the litres on board and the weight the drone carries in kilograms.

`drones.txt`

Each line describes one drone, with its values separated by commas:

```
<name>,<payload>,<purchaseDate>,<colour>,<bay>
```

- **name:** the drone's name.
- **payload:** the litres of water the drone carries when it leaves its bay.
- **purchaseDate:** the date the drone was bought, in the format `yyyy/MM/dd`.
- **colour:** the drone's colour, stored as a whole number.
- **bay:** the location of the drone's docking bay, always in row 0.

For example:

```
Lovelace,40,2025/03/12,16738816,C0
```

Lovelace carries 40 litres, was bought on 12 March 2025, has the colour `16738816` (which is `#FF6A00` in hex), and docks at `C0`.

The first three lines of `drones.txt` are:

```
Turing,5,2026/01/01,1238512,A0
Lovelace,40,2025/03/12,16738816,C0
Hopper,60,2021/06/30,14922561,E0
```

`jobs.txt`

Each line describes one field that needs water: its location and the litres it needs, separated by a semicolon.

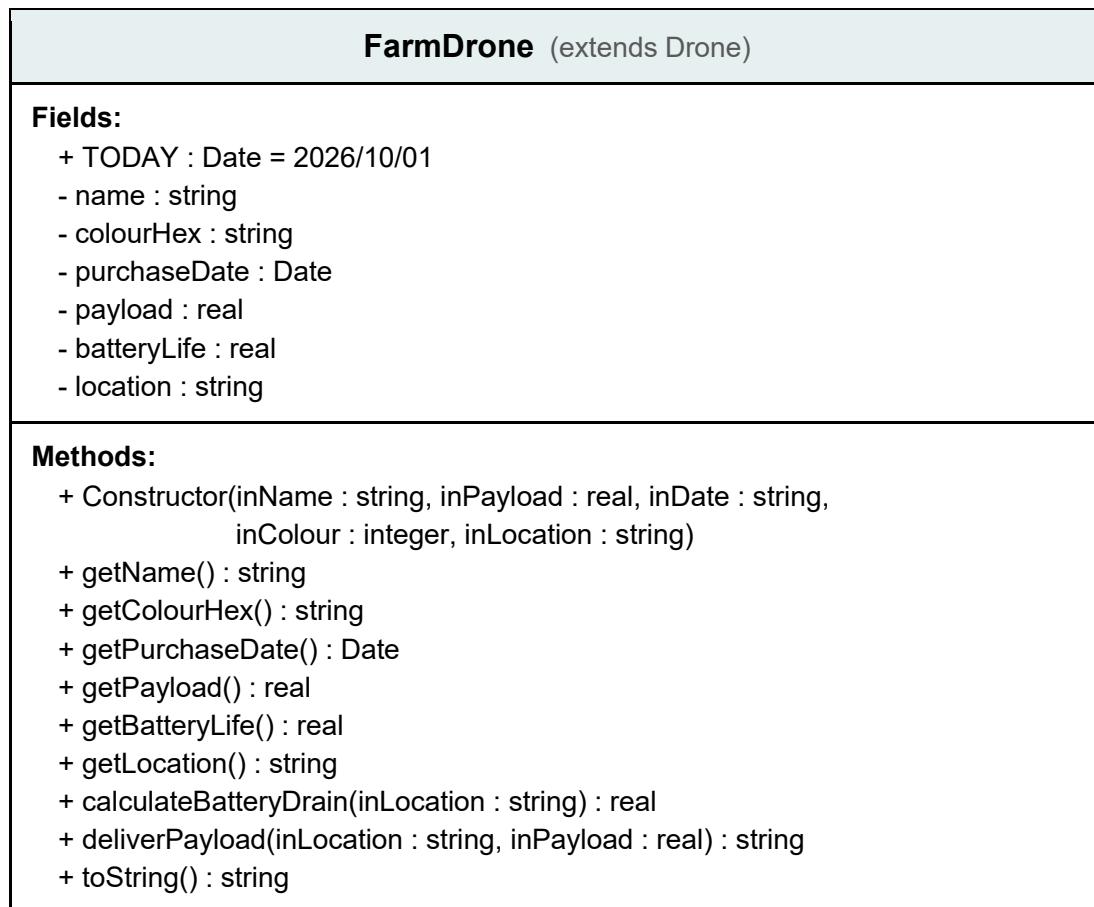
```
<location>;<litres>
```

The first three lines of `jobs.txt` are:

```
A6;5
D3;10
A9;25
```

Question 1: The FarmDrone class

Use the class diagram below to create a class called `FarmDrone`. The class stores the details of one drone and extends the provided `Drone` class.



- 1.1 Create a class called `FarmDrone` that extends the `Drone` class from the `exam.internal` package.
- 1.2 Add a public static constant called `TODAY`, which stores the date 1 October 2026. Whenever your program needs today's date, it uses `TODAY` rather than the computer's clock.
- 1.3 Add the fields `name`, `colourHex`, `purchaseDate`, `payload`, `batteryLife` and `location`, as shown in the class diagram. These fields must not be accessible outside the class.

1.4 Code a parameterised constructor that assigns its parameters to the fields, as follows:

- `inName`, `inPayload` and `inLocation` are assigned to `name`, `payload` and `location`.
- `inDate` is a string in the format `yyyy/MM/dd`. Convert it into a date before assigning it to `purchaseDate`.
- `inColour` is the colour as a whole number. Convert it into a hex string before assigning it to `colourHex`. The hex string starts with `#`, is followed by exactly six hex digits, and uses capital letters. For example, `1238512` becomes `#12E5F0`, and `65280` becomes `#00FF00`.
- Every drone starts with a full battery, so `batteryLife` starts at `100`.

1.5 Code an accessor method for each of the six fields. Each one overrides the matching method in `Drone`.

1.6 Code a `toString` method that returns the drone's name, colour, location and payload, separated by tabs:

```
name<tab>colourHex<tab>location<tab>payload
```

For example:

```
Turing      #12E5F0    A0    5.0
```

Note: `calculateBatteryDrain` and `deliverPayload` are coded in Question 4.

Question 2: The DroneManager class

- 2.1** Create a class called `DroneManager` that extends the `Manager` class from the `exam.internal` package.
- 2.2** Add two fields to the class. These fields must not be accessible outside the class.
- An array called `drones` that stores up to 10 drone objects.
 - A counter called `n` that counts the drones added to the array.
- 2.3** Code a constructor that reads the text file `drones.txt`. The constructor must:
- Read each line of the file and split it into its values.
 - Create a `FarmDrone` object from those values.
 - Add the object to the next available position in the `drones` array.
 - Increase the counter `n`.
- 2.4** Code a `toString` method that returns the details of every drone in the array, each on a new line. Use the `toString` method you coded in Question 1.6.

Question 3: The DroneUI class

3.1 Create a class called `DroneUI` with a `main` method.

3.2 Declare and instantiate a `DroneManager` object.

3.3 Call the `launch` method of your `DroneManager` object. This opens the farm window.

The window draws your fleet from the text your `toString` method returns. Each drone should be parked in its bay, in its own colour, with its name beneath it. The panel on the right lists each line of your `toString`, with a coloured dot beside it:

- **Green:** the line is correct.
- **Amber:** the line can be read, but its values disagree with `drones.txt`. The reason is shown under the line.
- **Red:** the line cannot be read, so that drone is missing from its bay. The reason is shown under the line.

The panel shows each tab as `\t`, so you can check that your values are separated by tabs and not spaces. The correct output from your `toString` is:

```
Turing      #12E5F0    A0    5.0
Lovelace    #FF6A00    C0    40.0
Hopper      #E3B341    E0    60.0
Babbage     #A371F7    G0    30.0
Dijkstra    #F778BA    I0    50.0
Knuth       #56D364    K0    45.0
Hamilton    #58A6FF    M0    35.0
```

When all seven lines are green, your drones are ready to fly.

Question 4: Flying a FarmDrone

In the `FarmDrone` class, code the two methods below. Code them after the methods from Question 1.

How far a flight is

Every field on the farm is a square, 100 metres wide and 100 metres long. Picture each location as a point on a plan of the farm:

- Moving one column to the right, such as from `A` to `B`, moves 100 metres across.
- Moving one row down, such as from row 3 to row 4, moves 100 metres down.
- Location `A0` is at 0 metres across and 0 metres down.

A drone flies in a straight line. If one location is `x1` metres across and `y1` metres down, and another is `x2` metres across and `y2` metres down, the distance between them in metres is:

$$\text{distance} = \sqrt{(\text{x2} - \text{x1})^2 + (\text{y2} - \text{y1})^2}$$

For example, `C0` is 200 metres across and 0 metres down, and `D2` is 300 metres across and 200 metres down. The distance between them is $\sqrt{(100^2 + 200^2)}$, which is about 223.6 metres.

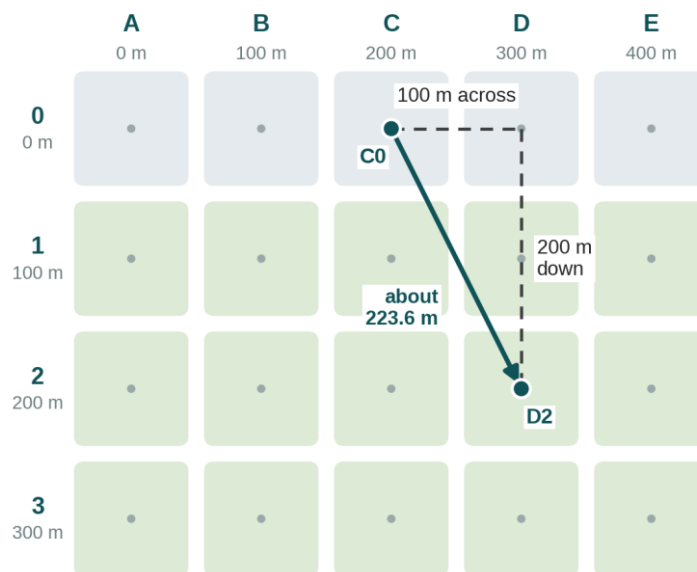


Figure 2: A flight from `C0` to `D2` covers 100 metres across and 200 metres down.

How much battery a flight uses

A drone's battery drain depends on three things: how far it flies, how much water it carries, and its age.

A drone's age is the number of complete years between its purchase date and `TODAY`. The age sets an age factor:

Age	Age factor
under 2 years	1.0
2 to 4 years	1.25
5 years or more	1.5

The battery drain, as a percentage, is:

```
drain = (distance / 100) × ageFactor × (1 + payload / 20)
```

where `distance` is in metres and `payload` is the litres on board before the flight.

4.1 Code the method `calculateBatteryDrain`. It accepts the location a flight goes to, and returns the battery percentage that a flight from the drone's current location to that location would use.

This method only works out the cost of a flight. The drone's location, payload and battery must be the same after the method runs as they were before it.

For example, Lovelace is 1 year old, carries 40 litres and is at `C0`. A flight to `D2` covers about 223.6 metres, so:

```
drain = (223.6 / 100) × 1.0 × (1 + 40 / 20) = about 6.7
```

4.2 Code the method `deliverPayload`. It accepts a location and the litres of water to release there, and it:

- Reduces the battery by the drain for the flight to that location. Work out the drain first, before anything else about the drone changes, since the drone carries all its water the whole way there.
- Reduces the payload by the litres released.
- Sets the drone's location to the new location.
- Returns a string with the drone's name, the location and the litres, separated by semicolons.

```
name;location;litres
```

For example:

```
Lovelace;D2;20.0
```

The drone always obeys. `deliverPayload` does not check whether the drone has enough battery or enough water, so a drone sent on a job it cannot manage ends up with a negative battery or a negative payload. Checking first is the job of the `DroneManager`, in Question 5.

Question 5: Watering the farm

In the `DroneManager` class, code the three methods below.

- 5.1** Code a helper method called `canTravel`. It accepts a drone, a location and a number of litres, and returns `true` only if the drone can do that job:
- It has enough battery for the flight. A flight that leaves the battery at exactly 0 is fine. One that would take it below 0 is not.
 - It has at least that many litres of water on board.
- 5.2** Code a method called `assignJob`. It accepts a location and the litres that field needs, and:
- Searches the `drones` array, in order, for the first drone that can do the job.
 - Sends that drone to deliver the water, and returns the string that `deliverPayload` returns.
 - Returns an empty string if no drone can do the job.
- 5.3** Code a method called `processJobs`. It reads the text file `jobs.txt`, and for each line:
- Splits the line into the location and the litres.
 - Calls `assignJob` to have a drone water the field.

The method returns every non-empty result from `assignJob` joined into one string, each on a new line. This string is the script that the display plays back.

5.4 Run your program. The display now plays your script one line at a time, and each delivery appears in the panel on the right as it happens:

- **Green:** the delivery worked.
- **Amber:** the drone arrived, but the field did not get the water it needed. The reason is shown under the line.
- **Red:** the drone crashed, or the line could not be flown. The reason is shown under the line.

The display does not trust your drones' numbers. It flies its own copy of every drone, using the formula from Question 4, so a slip in your battery calculation shows up as a drone falling out of the sky.

When the last drone has finished, a banner says whether you passed. You pass when every field has received all the water it needs. Click anywhere in the window to watch it again.

The correct script is:

```
Turing;A6;5.0
Lovelace;D3;10.0
Lovelace;A9;25.0
Hopper;I11;20.0
Babbage;B11;25.0
Dijkstra;I2;20.0
Dijkstra;G3;15.0
Lovelace;C4;5.0
Knuth;A8;25.0
Hopper;F8;15.0
Hamilton;M1;25.0
Dijkstra;D12;15.0
Knuth;L5;15.0
Hamilton;B5;10.0
```

END OF EXAMINATION